



Azure SEV-SNP attestation under the paravisor — a research brief

Prepared by extro for dvd, 2026-05-18. This is a gpt-5.5-pro deep-research synthesis (web-sourced, ~40 searches) assembled the same evening you flagged the `/dev/sev-guest` blocker. Sources are tagged by provenance at the end, weighted toward the non-Microsoft corroboration you wanted. You're the one who'll know what holds up on real hardware — treat this as leads and a runbook to validate, not gospel.

Verdict — does an AMD-root-of-trust attestation survive on Azure?

Yes, with one load-bearing caveat. Azure does expose an AMD-signed SEV-SNP hardware report that verifies to the AMD VCEK/VLEK → AMD CA chain *without* using Microsoft Azure Attestation (MAA) as the verifier — but it is delivered as a **paravisor/vTPM (HCL) report stored in the vTPM**, not a guest-triggered report via `/dev/sev-guest` with arbitrary guest `REPORT_DATA`.

So a customer-side verifier can root trust in AMD silicon and refuse to trust Microsoft as the *attestation authority*. It cannot remove Microsoft's HCL/OpenHCL/UEFI/vTPM code from the *measured TCB*. Freshness/nonce binding has to come off a **vTPM quote** whose AK is bound by the AMD report, not from guest-chosen AMD `REPORT_DATA`.

How attestation actually works on an Azure CVM

Where the paravisor sits. Azure confidential VMs boot with a Microsoft paravisor (OpenHCL) running in the encrypted guest context (VTL2; the guest OS in VTL0). It hosts device emulation and the vTPM. The Linux guest therefore does **not** see the normal SNP guest firmware device — stock `snpguest` `report`, `sevctl`, and generic TSM/configs flows that rely on `/dev/sev-guest` (`SNP_GET_REPORT`) are not the Azure path today.

The raw report lives in the vTPM, not at an IMDS endpoint. Azure-reserved NV indexes:

Item	NV index	Meaning
HCL attestation report	0x01400001	Azure HCLA blob with embedded AMD SEV-SNP hardware report
Report Data	0x01400002	64-byte data in HCL runtime data / claims
vTPM AK cert	0x01C101D0	Azure-issued AK certificate
vTPM AK handle	0x81000003	vTPM Attestation Key used to sign TPM quotes

HCLA blob layout: 32-byte Azure header, then the hardware report at offset 32 (for AMD SEV-SNP that payload is 1184 bytes), then runtime data at offset 1216 (JSON claims including the vTPM AK public key and VM config).

```

sudo tpm2_nvread -C o 0x01400001 -s 2600 -o hcl_report.bin
dd if=hcl_report.bin of=amd_report.bin bs=1 skip=32 count=1184 status=none
python3 - <<'PY'
import struct
b = open("hcl_report.bin","rb").read()
sig, ver, size, req_type, status = struct.unpack_from("<IIIII", b, 0)
print(f"sig={sig:#x} ver={ver} size={size} req_type={req_type} status={status} len={len(b)}")
print("expect sig 0x414c4348 == 'HCLA', version 2, request_type 2")
PY

```

IMDS is for certs and metadata, not the report.

- <http://169.254.169.254/metadata/THIM/amd/certification> → VCEK cert + chain (useful as a cert distribution channel, not the attestation authority; a strict verifier should be able to fetch AMD KDS itself or pin AMD ARK/ASK).
- <http://169.254.169.254/metadata/attested/document?api-version=2025-04-07&nonce=...> → a Microsoft PKCS#7-signed **metadata** document (VM ID, SKU, subscription, timestamp, nonce). Proves "this came from Azure." It is **not** the AMD SEV-SNP report and must not be the key-release root.

What MAA / AttestationClient does. `azguestattestation1 / AttestationClient` / Key Vault Secure Key Release gather guest evidence and return an **MAA-signed JWT**. Operationally clean, but the JWT issuer is MAA, i.e. Microsoft becomes the release authority. Use MAA only as a debugging oracle / comparison target, not the production gate.

What the AMD report signs on Azure. The embedded AMD report is still AMD-signed and verifies under VCEK/VLEK → AMD CA roots. Azure binds the HCL runtime data into the report via `REPORT_DATA` (the report's `report_data` is a hash of the runtime claims; the independent `az-snp-vtpm` crate validates exactly that the HCL variable-data hash equals the embedded SNP report's `report_data`). The runtime data carries the **vTPM AK public key** — so freshness is "AMD report binds vTPM AK; vTPM AK signs a fresh TPM quote containing your nonce / guest public-key binding." Standard layered attestation (RATS evidence/appraisal terms apply).

Measurement. The AMD report's launch measurement covers HCL/OpenHCL/UEFI/paravisor state, **not** the full guest image. Microsoft's own FAQ warns not to use `x-ms-sevsnpvm-launchmeasurement` as guest-code integrity. Guest OS / kernel / initrd / Secure Boot / app state must be verified through the **vTPM quote, PCRs, event log, and ideally IMA/dm-verity**.

The viable path

A composite Azure SNP-vTPM evidence bundle, verified by the customer device:

```

AMD ARK -> ASK/ASVK -> VCEK/VLEK -> AMD SEV-SNP report signature
report_data = H(HCL runtime data)
measurement = HCL/OpenHCL/UEFI/paravisor launch state
HCL runtime data -> contains vTPM AK public key + VM config
vTPM AK -> signs TPM quote over: PCR digest + H(nonce || guest_ephemeral_pubkey || ctx)
PCRs + event log + optional IMA/dm-verity -> establish the guest image / boot / app policy
Customer verifier -> if all checks pass, releases the LUKS key to guest_ephemeral_pubkey

```

1. Guest reads the HCLA blob from vTPM NV `0x01400001`; keep the full blob (the verifier needs the runtime data, not just the 1184-byte AMD report).

2. Verifier validates the AMD chain (VCEK/VLEK → ARK/ASK), report signature, TCB/security-version policy, debug bit, VMPL, SNP policy, product, revocation.
3. Verifier validates the HCLA binding: hash the runtime data per the HCLA Hash Type, compare to the AMD report REPORT_DATA, extract HCLAKPub from runtime claims (this is why you don't have to trust Azure's AK cert as a root — the AK is trusted because it's inside AMD-bound runtime data).
4. Guest makes the key-release binding: generate an ephemeral X25519/TLS key, take a fresh verifier nonce, ask the vTPM AK to sign a TPM quote whose qualifying data is `H("sol-key-release-v1" || nonce || guest_pubkey || ctx)`; the quote also covers measured-boot PCRs.
5. Verifier validates the quote under HCLAKPub, checks extraData == the binding hash, checks PCR digest + event log + Secure Boot/TPM config + image/app measurements, then releases the LUKS key encrypted to the quoted guest public key.
6. Guest mounts the separate encrypted data disk; the OS image stays non-secret; the key only ever exists in the verifier and in guest confidential memory.

This is **not** byte-for-byte portable with bare-metal/AWS/GCP raw `/dev/sev-guest`. A portable seam needs two evidence types — `AmdSnpRawReport` and `AzureSnpVtpmCompositeEvidence` — normalized to common claims (AMD-root validity, freshness binding, TCB, measurements, policy) with an Azure-specific verifier plugin. ("A little extra Azure customization unifies it" is roughly right at the *claims* layer, not the evidence-parser layer.)

What's possible now vs blocked vs needs-Microsoft

Capability	Status	Note
Obtain AMD-signed SNP report on Azure Linux CVM	works today	vTPM NV 0x01400001; extract 1184-byte AMD report
Stock <code>/dev/sev-guest</code> , <code>sevctl</code> , plain <code>snpquest</code> report	blocked on Azure	native SNP ioctl path not exposed
<code>snpquest</code> on Azure	works with effort	build <code>VirTEE snpquest --features hyperv; snpquest report ... --platform</code> . Azure support landed in 2025-era releases
Verify AMD report to AMD root without MAA	works today	raw report + VCEK/VLEK/CA from AMD KDS or THIM
Avoid Microsoft as attestation authority	works today	don't use MAA JWT as the release token; verifier appraises raw AMD report + vTPM quote
Avoid Microsoft code in the measured TCB	blocked on Azure	HCL/OpenHCL/UEFI/vTPM are in the attested TCB; you can measure/pin, not remove
Guest-chosen nonce/pubkey in AMD <code>REPORT_DATA</code>	blocked / confirm w/ MS	Azure report is pre-generated/HCL-mediated; use TPM-quote freshness instead
Bind key release to nonce/pubkey	works with effort	bind in TPM quote qualifying data, AK rooted via AMD-bound HCLA runtime data
Pin guest image via AMD launch measurement alone	wrong layer	AMD measurement = HCL/UEFI/paravisor; guest integrity is PCR/ event-log/IMA/dm-verity
Pin guest image via vTPM PCRs	works with effort	needs stable measured-boot policy, event-log validation, eventually IMA/dm-verity
IMDS <code>/metadata/attested/document</code> as AMD attestation	wrong artifact	MS PKCS#7 metadata, not SNP evidence
IMDS <code>/metadata/THIM/amd/certification</code>	works today	cert retrieval only
Compute expected Azure HCL launch measurement offline	blocked / needs MS reference data	HCL/IGVM/UEFI/config are MS-controlled; need published reference values or empirical allowlists
<code>configs-<i>tsm</i></code> path on Azure	needs MS/kernel confirm	good generic direction; Azure default CVMs currently expose vTPM/HCLA
Mount separate DM-Crypt/LUKS data disk post-attestation	works today	orthogonal to Azure confidential OS disk; release key, then <code>cryptsetup open</code>

Ordered runbook

1. **Use a known-good CVM image first** (don't debug on arbitrary marketplace images). Ubuntu 22.04 CVM on DCasv5/DCadsv5 or ECasv5/ECadsv5; move to v6 once quota/image compat is clear.

```
LOC=eastus2; RG=sol-cvm-attest-rg; VM=sol-cvm-attest-01
SIZE=Standard_DC2as_v5
IMAGE=Canonical:0001-com-ubuntu-confidential-vm-jammy:22_04-lts-cvm:latest
az group create -n "$RG" -l "$LOC"
az vm create -g "$RG" -n "$VM" -l "$LOC" --size "$SIZE" --image "$IMAGE" \
  --security-type ConfidentialVM --enable-vtpm true --enable-secure-boot true \
  --os-disk-security-encryption-type VMGuestStateOnly \
  --admin-username azureuser --ssh-key-values ~/.ssh/id_ed25519.pub
```

SKU / quota discovery:

```
az vm list-skus --location "$LOC" --resource-type virtualMachines --all \
  --query "[?capabilities[?name=='ConfidentialComputingType']].\
    {Name:name,Family:family,CC:capabilities[?name=='ConfidentialComputingType'].value|\
    [0],Restrictions:restrictions}" -o table
az vm list-usage --location "$LOC" -o table
```

2. Confirm the shape inside the VM.

```
systemd-detect-virt --cvm || true
sudo dmesg | egrep -i 'sev|snp|vtom|tpm|hcl|hyper-v' || true
ls -l /dev/sev-guest /dev/tpm* 2>/dev/null || true
```

Expect: no usable /dev/sev-guest; yes /dev/tpmrm0//dev/tpm0.

3. Tooling.

```
sudo apt-get update
sudo apt-get install -y build-essential git curl jq tpm2-tools pkg-config \
  libssl-dev xxd python3 cryptsetup
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
. "$HOME/.cargo/env"
```

4. Pull HCLA + extract raw AMD report (see the tpm2_nvread/dd/python snippet above; assert HCLA sig 0x414c4348, version 2, request_type 2).

5. Build snpguest with Azure support and cross-check.

```
git clone https://github.com/virtee/snpguest && cd snpguest
cargo build -r --features hyperv
sudo install -m 0755 target/release/snpguest /usr/local/bin/snpguest
sudo snpguest report snpguest_amd_report.bin platform_report_data.bin --platform
snpguest display report snpguest_amd_report.bin | head -100
```

Diff amd_report.bin vs snpguest_amd_report.bin; if they match, standardize on snpguest --platform for raw-report extraction and keep tpm2_nvread for HCLA runtime parsing. (This is likely the "patch" you were eyeing — the key piece is already upstream; the useful contribution is evidence-bundling, not emulating /dev/sev-guest.)

6. Fetch AMD cert material and verify the raw report.

```

snpguest fetch ca pem certs --report amd_report.bin -e vcek
snpguest fetch vcek pem certs amd_report.bin
snpguest verify certs certs
snpguest verify attestation certs amd_report.bin --signature
snpguest verify attestation certs amd_report.bin
# Azure THIM as a cert distribution channel (treat as cache, not authority):
curl -fsS -H Metadata:true --no-proxy "*" \
  "http://169.254.169.254/metadata/THIM/amd/certification" | tee thim.json
jq -r '.vcekCert, .certificateChain' thim.json > thim_vcek_chain.pem

```

7. **Validate the HCLA runtime-data binding, not just the AMD signature.** `az-snp-vtpm` is the fastest independent verifier reference (retrieves the vTPM report, parses HCLA, validates the AMD chain, checks HCL variable-data hash == SNP report_data):

```

git clone https://github.com/kinvolk/azure-cvm-tooling && cd azure-cvm-tooling
cargo build --release -p az-snp-vtpm
sudo ./target/release/snp-vtpm -p

```

If it passes, port the parsing/verification logic into your verifier rather than depending on Microsoft's AttestationClient.

8. **Nonce/public-key binding via a vTPM quote.**

```

sudo openssl genpkey -algorithm X25519 -out guest_x25519.key
sudo openssl pkey -in guest_x25519.key -pubout -outform DER -out guest_x25519.pub.der
NONCE_HEX="$(openssl rand -hex 32)" # in production: from the customer device
BINDING_HEX="$( { printf 'sol-key-release-v1'; printf '%s' "$NONCE_HEX" | xxd -r -p; \
  sudo cat guest_x25519.pub.der; } | openssl dgst -sha256 -binary | xxd -p -c 256 )"
sudo tpm2_readpublic -c 0x81000003 -f pem -o akpub.pem
sudo tpm2_quote -c 0x81000003 -l sha256:0,2,4,7,8,9,15,16,22,23 \
  -q "$BINDING_HEX" -m quote.msg -s quote.sig -o quote.pcrs -g sha256
sudo cp /sys/kernel/security/tpm0/binary_bios_measurements eventlog.bin 2>/dev/null || true

```

Verifier-side sanity check:

```
tpm2_checkquote -u akpub.pem -m quote.msg -s quote.sig -f quote.pcrs -g sha256 -q
"$BINDING_HEX"
```

— but in the real flow verify the quote with `HCLAKPub` extracted from HCLA runtime data, not merely an Azure-issued AK cert.

9. **Verifier policy (minimum, even if the first PCR policy is permissive):** AMD report signature valid · VCEK/VLEK chains to pinned AMD roots · TCB/security version acceptable · SNP debug disabled · VMPL/policy acceptable · HCLA runtime-data hash == AMD REPORT_DATA · `HCLAKPub` extracted from runtime claims · TPM quote valid under `HCLAKPub` · quote extraData == binding hash · PCR digest matches · PCR/event-log policy matches the expected image · Secure Boot/TPM config acceptable · `vmUniqueId/disk/deployment` identity matches the expected instance.

10. **Attach + mount the separate encrypted data disk.**

```

az disk create -g "$RG" -n "${VM}-data-01" -l "$LOC" --size-gb 64 --sku Premium_LRS
az vm disk attach -g "$RG" --vm-name "$VM" --name "${VM}-data-01" --lun 0
# in guest, after the verifier releases the key:
ls -l /dev/disk/azure/ 2>/dev/null; lsblk -o NAME,SIZE,TYPE,MOUNTPOINTS,MODEL
DISK=/dev/disk/azure/scsi1/lun0 # adjust after lsblk
sudo cryptsetup luksFormat --type luks2 --batch-mode "$DISK" /run/sol-keys/journal.luks.key
sudo cryptsetup open "$DISK" sol_journal --key-file /run/sol-keys/journal.luks.key
sudo mkfs.ext4 -L sol_journal /dev/mapper/sol_journal
sudo mkdir -p /var/lib/sol/journal && sudo mount /dev/mapper/sol_journal /var/lib/sol/journal

```

1. **Don't patch sevctl.** If anything, the contribution surface is `snpquest/verifier` evidence bundling (emit both `hcl_report.bin` and `amd_report.bin`, parse HCLA runtime claims, verify the runtime-data hash vs AMD `REPORT_DATA`, optionally emit a composite bundle with quote / PCRs / event log). Don't try to emulate `/dev/sev-guest` inside the Azure guest.

Open questions worth confirming with Microsoft (public sources don't close these)

1. Is there any supported/planned Azure CVM API (OpenHCL, SVSM, configs-tsm, ioctl, vTPM command) to request a **fresh AMD SEV-SNP report with caller-supplied 64-byte REPORT_DATA** — or is vTPM-quote chaining the intended freshness mechanism by design?
2. Where are production Azure HCL/OpenHCL/UEFI measurement reference values, build provenance, and update cadence published, so an offline verifier can pin them **without MAA**?
3. What is the stable Linux PCR allocation for Azure CVMs (kernel/initrd/ cmdline, Secure Boot, vm-configuration, recommended app/IMA PCR)?
4. Do NonPersistedTPM, DCasv6/ECasv6, or newer OpenHCL change the HCLA report lifecycle, vTPM AK persistence, or availability of dynamic report generation?

Risks / where the clean model has to bend

1. **Guest-chosen REPORT_DATA doesn't survive on Azure default CVMs.** Freshness moves to a vTPM quote whose AK is AMD-bound. Cryptographically sound, just not the literal raw-SNP mechanism.
2. **No-Microsoft-as-authority is achievable; no-Microsoft-in-TCB is not.** Avoiding MAA removes Microsoft as verifier/signer of the release decision; it does not remove HCL/OpenHCL/UEFI/vTPM from the measured TCB.
3. **Reference values are the hard problem.** Verifying a signature is easy; deciding which HCL/UEFI/PCR/event-log/kernel/app measurements are acceptable is the real policy. Without MS-published HCL reference values you either pin observed measurements empirically (fine for engineering, brittle for a customer-facing guarantee) or accept MAA/compliance claims.
4. **Guest image integrity isn't in the raw AMD measurement** — pin it via vTPM PCR/event-log policy, and add dm-verity/IMA/systemd measurement so a compromised app can't extend a PCR and then run different code.
5. **Plain HTTP to 169.254.169.254 isn't the main risk here** — IMDS is only a cert/metadata channel; the AMD report and TPM quote are self-verifying. A local HTTP attacker can DoS or confuse a sloppy client, not forge an AMD-root-valid report. Still: `--noproxy "*" , strict parsing, verifier-side AMD-root pinning.`

Sources (tagged by provenance)

- **[primary]** AMD SEV-SNP attestation / firmware ABI — report semantics, fields, key hierarchy. amd.com (lss-snp-attestation.pdf).
- **[primary]** Linux kernel SEV-guest API docs — confirms the normal non-Azure path (`/dev/sev-guest`, `SNP_GET_REPORT`); Azure's absence of the device is an Azure distinction, not a generic Linux fact. kernel.org.
- **[independent / OSS]** VirTEE `snpguest` README — current Azure support via `--platform + --features hyperv`; states the paravisor stores the report in the vTPM NV index and hides `/dev/sev-guest`. github.com/virtee/snpguest.
- **[independent / OSS]** `az-snp-vtpm` / Kinvolk `azure-cvm-tooling` — corroborates the HCLA/vTPM verification model and nonce-via-quote freshness. docs.rs/az-snp-vtpm.
- **[MS / OSS docs]** OpenHCL / OpenVMM guide — paravisor role, VTL2/VTL0, Azure Boost, IGVM packaging. openvmm.dev.
- **[Microsoft]** Azure CVM guest-attestation design — vTPM evidence model, NV indexes, HCLA format, runtime claims, layer coverage. learn.microsoft.com.
- **[Microsoft]** Azure vTPM how-to / guest-attestation FAQ — the `tpm2_nvread`/THIM recipes; explicitly: report is not dynamically generated, measures HCL/UEFI. (MS-sourced but corroborated by VirTEE tooling.)
- **[Microsoft]** Azure IMDS docs — `/metadata/attested/document` is MS PKCS#7 metadata, not an SNP report. learn.microsoft.com.
- **[Microsoft]** MAA / Secure Key Release docs — the managed AttestationClient → MAA JWT → Key Vault SKR flow (useful for dev; not the customer-verifier gate). learn.microsoft.com.
- **[primary / standards]** IETF RATS RFC 9334 — attester/evidence/verifier/ endorsements/appraisal/ freshness vocabulary. ietf.org.
- **[independent / vendor]** Canonical Ubuntu-on-Azure docs — CVM support; full-disk encryption optional relative to the TEE. documentation.ubuntu.com.

Flagged: any claim about current Azure production behavior ultimately rests partly on Microsoft's own documentation; where that's the case it is corroborated by the independent VirTEE / az-snp-vtpm tooling, which is the strongest non-Microsoft confirmation available today.